

Sr.No. Date

Topic

Sign

1

Introduction of Big Data

1. Explain three Vs of Big Data.

→ Definition of Big Data:

Big Data is a term used to describe data that has massive volume, comes in a variety of structures and is generated at high velocity. This kind of data poses challenges to the traditional RDBMS system used for storing & processing data.

The three Vs of Big Data :-

1) Volume :

Volume in Big Data means the size of the data. As businesses become more transaction-oriented, we see ever increasing no. of transactions. More devices are getting connected to internet, which is adding to volume. This huge volume of data is biggest challenge for big data technologies. The storage and processing power needed to store, process & make accessible data in timely & cost efficient manner.

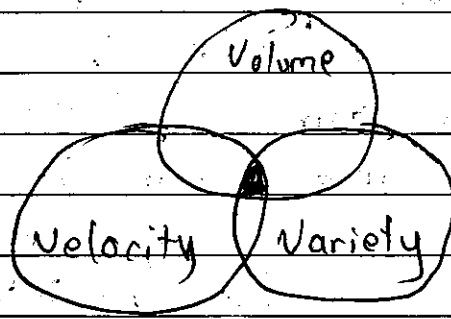
2) Variety :

The data generated from various devices & sources follows no fixed format structure. Varieties of big data: Text & Log files, stream videos, photos & various other unstructured formats. There is no control over data structure these days. New sources & data structures are being created rapidly.

3) Velocity :

Velocity in big data is speed at which data is created & at which it's needed to be processed. If data cannot

processed at needed speed, it loses its significance. Reacting & processing quickly enough to deal with volume of data is one more challenge for a big data tech.



Q.2 What is MongoDB design philosophy? Explain.
→ MongoDB is a document-oriented NoSQL Database created by MongoDB Inc. in 2007. It was designed around 5 core philosophical principles:

1. Speed, Scalability and Agility:
MongoDB was built from ground up to be fast, horizontally scalable and agile. Traditional RDBMS systems need schema migrations for any structural change. MongoDB allows devs to modify data structure without delay, enabling rapid app development and evolution. Horizontal scaling allows MongoDB to handle growing data volumes cost-effectively.

2. Non-Relational Approach:
MongoDB uses document-oriented model. Each record is a self-contained doc in BSON/JSON format that can have unique structure. No JOIN are made at DB level, no foreign key constraints and no rigid relational schema. Related data is directly embedded within a doc.

3. JSON-based document store: pto →

MongoDB stores data internally as BSON but shows it to devs as JSON. JSON is dominant data format for modern web APIs & apps, making MongoDB a natural fit of devs. Does support nested objects & arrays.

4. Performance v/s Features:

MongoDB deliberately uses raw performance & scalability over advanced ADBMS features like multi-table JOINS, ACID transactions. Idea is to give speed & scale to devs & let complex business logic be handled at app-level.

5. Running Database Anywhere:

MongoDB is designed to run on commodity hardware rather than expensive specialized hardware. It can be deployed on-premise, cloud or in hybrid environments. This reduces infra cost.

Q. 3.

Briefly explain CAP Theorem.

AKA Brewer's theorem, was proposed by computer scientist Eric Brewer in 2000, the CAP states that a distributed data system can only guarantee TWO out of full 3 properties at any given time.

- Consistency (C): Every read receives most recent write or an error. All nodes in distributed sys. see same data at same time. There are no stale reads - every request gets current, up-to-date data.

- Availability (A): Every request receives a response, even if not recent. The sys. accepts every valid request. It prioritizes staying operational over guaranteeing up-to-date data.

- Partition Tolerance (P): The sys. continues to operate even if some nodes are unreachable.

network partitions occur - i.e. when some nodes in distributed sys. cannot communicate with others due to network failure

Practical Implication:

System Type	Guarantees	Sacrifices	Examples
CP	Consistency + Partition Tolerance	Availability during failure	MongoDB, HBase
AP	Availability + Partition Tolerance	Immediate consistency (Eventual)	Cassandra, CouchDB
CA	Consistency + Availability	Partition Tolerance (Not Distributed)	MySQL, PostgreSQL

Q. 4. Differentiate between SQL and NoSQL Databases.

Aspect	SQL (Relational DB)	NoSQL (Non-Relational DB)
Full form	Structured Query Language	Not only SQL
Data model	Tables with rows and columns	Document, Key-value, Column-family or graph
Schema	Fixed, predefined (rigid)	Dynamic, flexible (Schema-less)
Scalability	Vertical - scale up (more powerful server)	Horizontal (more servers) - scale out
Consistency Model	Strong - ACID properties guaranteed	Eventual - follows BASE model
Query language	Standardised SQL across all systems	Varies per database (no universal standard)
Relationships	Foreign keys & JOINs	Embedded documents or references.

Transactions	Full multi-table ACID transactions	Limited (improving in newer versions)
Performance	Optimized for complex queries	Optimized for high volume read/write.
Example	MySQL, Oracle, SQL Server, PostgreSQL	MongoDB, Cassandra, Redis, HBase

- Q. 5. Write a short note on non-relational approach
- 1. This is one of core design philosophies of NoSQL and MongoDB databases.
2. It represents a fundamental departure from traditional relational model used in RDBMS like Oracle & MySQL.
3. In a relational DB, data is stored in tables with a fixed schema every row must conform to same column structure. Relationships are managed through foreign keys & JOIN.
4. Non-relational approach abandons this rigid structure in favour of a more flexible, document-oriented model.
5. As per no fixed schema, each doc. in a MongoDB collection can have a completely diff. structure. No need for uniformity.
6. No JOIN states that instead of joining multiple tables to get related data directly within a single document. This eliminates expensive JOIN op. & improves read performance.
7. Doc-oriented storage states that data is stored as self-contained docs, each representing a complete entity.
8. In Horizontal Scalability, non-relational DB can easily distribute data across many servers.
9. Flexible data modelling means devs can model data apps. naturally use it, leading to faster development & easier schema evolution.
10. Ex:- In a relational DB, storing a customer with multiple addresses need two tables. In MongoDB, single array is used within customer doc.

Q.6.
→

Discuss various applications of Big Data:

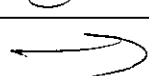
1. Visibility: Data accessibility in timely fashion to relevant stakeholders generates tremendous value. When data from many dept. & sources is integrated and made accessible in real time, org. can reduce search time & improve decision qual.
Ex:- Company with RnD, Engg., spread geographically - integrated data access improves product quality & reduces redundant work.
2. Discover & Analyze info: Most of Big Data's value comes from merging externally connected data with org.'s internal data. The data reveals new patterns & uncover imp. insights. Ex:- E-commerce companies merge purchase history, browsing behaviour, etc to identify buying factors & improve product recommendations.
3. Segmentation & Customisation: Big Data allows org. to create tailor-made products & services for specific customer segments.
Ex:- Streaming platforms like Netflix analyze viewing history to recommend personalised content.
4. Aiding Decision Making: Big Data analytics can substantially minimize risks, improve decision-making & uncover valuable insights. Automated systems make faster & more accurate decisions.
Ex:- Automated fraud alert systems in credit card processing detect unusual patterns instantly.
5. Innovation: Big Data enables innovation of new products, services & business models. Real-time data from machine vehicles & users provides insights that drive next gen of innovation.
Ex:- Real-time data from engines enables predictive maintenance, reducing breakdowns. Fuel consumption data leads to higher efficiency engine designs.

S.No. Date
2

Topic

Sign

Q.1. Explain Capped Collections.



1. It's a fixed size, circular collection.
2. This collection automatically overwrites (deletes) the oldest documents when the collection reaches its specific max. size or max. no. of docs.
3. It works like a circular buffer — new data comes in; old data goes out automatically.
4. This collection guarantees that docs are returned in the order they were inserted. No explicit sort is required.
5. ∴ docs are always appended at end of collection & there is no indexing overhead for deletion. capped collection offer very high write performance.
6. Updates to docs in this collection cannot increase the doc's size. If an update would cause a doc to grow, it will be rejected.
7. You cannot delete individual docs from this collection using ~~remove()~~ command. Only way is to drop the entire collection.

8. Apart from size, you can also specify a max. no. of docs. MongoDB enforces whichever limit is hit first.
9. It maintains the most recent N audit events for compliance without unbounded storage growth.
10. It stores the most recent N log entries without worrying about old logs filling up disk space.
11. A capped collection cannot be sharded.
12. The max. size must be specified at creation time & cannot be easily changed later.

Q.2. What are various tools available in MongoDB? Explain.

→ MongoDB provides a rich set of tools & processes to support DB administration, data management, monitoring and operations. These are broadly categorized into Core Processes & Utility Tools: —

A. Core Processes: —

• **mongod** (MongoDB Daemon) — The primary daemon process that runs MongoDB DB server. It handles all data requests, manages data access & is the heart of

MongoDB - without mongod running, no DB ops are possible.

- **mongo** (MongoDB shell) - An interactive JS command-line shell that allows developers & administrators to connect to MongoDB instance & run queries, commands & scripts. This is primary interface for interacting with MongoDB DB directly.
- **mongos** (MongoDB Sharding Router) - The routing service for sharded clusters. It acts as the query router, directing client requests to appropriate shard(s).

B. Backup & Restore Tools

- **mongodump** :- A utility that creates a binary export (backup) of contents of a MongoDB DB. It exports data in BSON format. Can backup an entire DB or specific collections.
- **mongorestore** :- Restores data from a BSON dump created by ~~Mongodump~~ **mongodump** back into a MongoDB instance.

C. Import & Export Tools:-

- **mongoexport** - Exports data from a MongoDB collection to a JSON or CSV file. Unlike **mongodump**, this produces human

readable output suitable for sharing or loading into other sys.

- `mongoimport` - Imports data from a JSON, CSV, or TSV file into a MongoDB collection. Useful for initial data loading & migration.

D. Monitoring Tools

- `mongostat` - Provides a real-time view of MongoDB server statistics, including insert, query, update, delete rates, connections, memory usage & similar to Unix 'vmstat'.
- `mongotop` - Tracks & reports amount of time MongoDB spends reading & writing data at collection level.

Ex: MongoDB Cloud Manager

It's MongoDB's hosted mgmt. platform that provides monitoring, backup, & automation for MongoDB deployments.

Q. 3.

Discuss points to be considered while importing data in a sharded environment.

→ In MongoDB sharding, data is distributed across multiple servers (shards) based on

a shard key. Importing data into such an environment requires special care to ensure data is distributed correctly & performance is maintained.

Key Points to Remember when importing data in a Sharded Environment:-

1. Choose correct shard key before importing:-
Shard key determines how data is distributed across shards. Once set, it cannot be changed for existing data. Poor key can lead to unbalanced distribution.

2. Import directly to correct shard when possible
For most efficient import, connect directly to mongos (the query router) rather than individual shards. Mongos will route data to correct shard based on key automatically.

3. Ensure Config Servers are running & healthy:-
Config servers store the cluster's metadata & chunk distribution info. They must be running & accessible during import. If Config servers are unavailable, mongos cannot route requests & import will fail.

4. Use mongoimport via mongos:-
Always use the mongoimport utility connecting

through the mongos router (not directly to a shard) to ensure data is properly routed & metadata is correctly updated.

5. Network bandwidth considerations :- Large imports in a sharded environment generate significant network traffic as data is routed through mongos & distributed to shards. Plan imports during off-peak hours & ensure sufficient network bandwidth is available.

6. Pre-split chunks before import :- When importing large volumes of data, it is recommended to manually pre-split chunks before importing. This prevents MongoDB from having to split chunks during the import process, which can cause performance degradation & uneven distribution.

Q. 4. Explain the concept of inserting by explicitly specifying `_id`.

→ The id Field :- In MongoDB, every document must have a unique identifier called `_id`. By default, MongoDB automatically generates a 12-byte ObjectId as the `_id` value when a document is inserted.

Default Behaviour (Auto-generated - id)

When you insert a doc. with specifying `_id`, MongoDB automatically assigns a unique `ObjectId`. An `ObjectId` is a 12-byte BSON type consisting of:

- 4 bytes - timestamp (sec. since Unix epoch)
- 3 bytes - machine identifier
- 2 bytes - process ID
- 3 bytes - random counter value

Explicitly Specifying `_id` :-

You can explicitly provide your own `_id` value during insertion. This overrides MongoDB's automatic `ObjectId` generation. The custom `_id` can be any BSON data type - integer, string, UUID or any other unique value.

Key Rules & Behaviours :-

• Uniqueness Enforced :- The `_id` field must be unique within a collection. If you attempt to insert a doc. with a `_id` that already exists, MongoDB throws a duplicate key error & rejects the insertion.

• `_id` cannot be updated :- Once a doc. is

inserted with a specific id it's permanent
Any attempt to update will throw an error

- Cannot be Null:- It must have a valid, non-null unique value at all times.

When to explicitly specify id.

- When migrating data from another DB & you want to preserve existing IDs.
- When you want human-readable IDs for debugging or logging.
- When integrating with external systems that use their own ID formats.

Q. 5. Discuss Indexes & its types:-

→

1. An index is a special data structure that stores a small portion of collection's data in an easy-to-traverse form.
2. Indexes support efficient execution of queries. Without an index, MongoDB must perform a collection scan to find matching docs.
3. With an index, MongoDB can limit the no. of docs it examines dramatically.
4. Indexes store the value of a specific

field / set of fields, ordered by value.
 MongoDB's indexes use B-tree data structure.

5. When a query is executed on an indexed field, MongoDB uses the index to locate matching docs quickly.

Types :-

- 1) Default `_id` index :-
 Created automatically when collection is created.
 Cannot be dropped.
 Ensures every doc has a unique one.

```
> db.collection.find({ id: ObjectId('...') })
```
- 2) Single Field Index :-
 Created on a single field of doc.
 Supports both ascending (1) and descending (-1) sort order.

```
> db.students.createIndex({ name: 1 })
```
- 3) Compound Index :-
 Index on many fields.
 Useful for queries that filter on multiple fields simultaneously & order of fields matters.

```
> db.students.createIndex({ name: 1, marks: -1 })
```

- 4) Multikkey Index :-
Created automatically when an index is created on a field that contains an array. Each value in array is indexed separately.
→ `db.products.createIndex({tags: 1})`
- 5) Text Index :-
Index that supports full-text search queries on string content. A collection can have at most 1 text index, but the text index can cover multiple fields.
→ `db.articles.createIndex({content: "text", title: "text"})`
- 6) Geospatial Index :-
Supports queries for location-based data. Used in location-aware apps. 2d & 2dsphere are two types.
→ `db.places.createIndex({location: "2dsphere"})`
- 7) Hashed Index :-
Indexes the hash of value of a field. Used primarily for hash-based sharding. Support equality queries but not range queries.
→ `db.users.createIndex({userId: "hashed"})`
- 8) Sparse Index :-
Only indexes docs that contain the indexed

field - Dogs, without the indexed field
are excluded from index entirely.
→ db.wxyz.createIndex({email: 1},
{sparse: true})

Q6. Write a short note on Data Distribution process-

1. When a collection is sharded, MongoDB distributes its docs across shards.
2. MongoDB divides entire shard key range value into chunks, assigned to a shard.

Components :-

- Shard Key:- Field on which data is partitioned. Choice of it's critical.
- Chunks:- MongoDB divides shard key value space into ranges as chunks.
- Config Servers:- Store cluster's meta data. This chunk map is consulted by mongos every time it routes a query to correct shard.
- mongos (Query Router):- The routing process that sits betⁿ app & shards. When query arrives, mongos consults config server's chunk map & routes the query to shard(s).

Importance :-

- Ensures no single ~~hard~~ shard becomes a bottleneck - load is spread across

- all available shards.
- Enables MongoDB to handle datasets far larger than single server's storage or memory capacity.
 - Transparent to app - the app connects to mongos & does not need to know which shard holds which data.

Steps :-

- 1) Shard Key Selection:- Admin selects shard key for collection.
- 2) Initial Chunk Creation:- MongoDB creates an initial chunk covering entire shard key range.
- 3) Chunk Splitting: As data is inserted & a chunk grows beyond max. chunk size, MongoDB automatically splits it into two smaller chunks. This splitting is handled by mongod process on shard where the chunk resides.
- 4) Chunk Migration:- After splitting, if one shard has significantly more chunks than others, balancer process kicks in & migrates chunks from overloaded to less loaded shard.
- 5) Metadata update:- After every split & migration, config servers are updated to reflect new chunk-to-shard mapping. mongos fetches this updated metadata to ensure future queries are routed correctly.